CNET101 – COMPUTER NETWORKS – SPRING 2026

GIU-Net Project

Expected Final Output

What a successful submission looks like — console output, captures, screenshots, and topology — without showing implementation.

SEQ

PAYLOAD

ACK

DATA

Prepared by the Teaching Team

What you are building, in one slide

PART 1

UDP Chat Application

- Custom protocol on raw UDP
- 2-step handshake (HELLO / OK)
- Sequence numbers + ACKs
- Retransmission with 3-second timeout
- Bi-directional chat across two machines
- Live Wireshark capture

PART 2

Campus Network Simulation

- Cisco Packet Tracer topology
- VLSM subnetting of a /22 block
- Static routing between two routers
- HTTP service + custom team webpage
- Simulation-mode hop-by-hop trace
- ACL bonus task

PART 1

UDP Chat — Expected Runtime Output

Console screens, captures, and logs — what graders will see on screen.

Server side — expected output

server-machine — bash

```
$ java GIUServer

[SERVER] Listening on port <your-derived-port>

[SERVER] Waiting for connection...

[SERVER] Received: HELLO-GIU-<LeaderID>-<Sum>

[SERVER] Handshake complete.

[SERVER] Connection established!

[RECEIVED] Hello from the client side

How are things on your end?

[RECEIVED] All good — another one

EXIT
```

Required tags

[SERVER]
lifecycle

[RECEIVED]
incoming DATA

[TIMEOUT]
after 3 s

[CONNECTION LOST]
give-up

[Connection closed by remote.]
peer EXIT

Client side — expected output

client-machine — bash

```
$ java GIUClient

[CLIENT] Type CONNECT to open a connection.

CONNECT

[CLIENT] HELLO sent: HELLO-GIU-<LeaderID>-<Sum>

[CLIENT] Server replied: OK-GIU-<LeaderID>-<Sum>

[CLIENT] Handshake complete.

[CLIENT] Connected! Type messages or EXIT.

Hello from the client side

[RECEIVED] How are things on your end?

All good — another one

[Connection closed by remote.]
```

Key checks

Socket not opened until the user types **CONNECT**

HELLO and OK strings include the team's actual leader ID and digit sum

Typing and receiving happen simultaneously — neither blocks the other

What timeout & retransmission look like

Scenario A — recovered after retry

no ACK on first try — ACK arrives on retry

```
Quick test message

[TIMEOUT] Retrying...

[RECEIVED] reply from peer
```

Sender waits up to 3 s, retransmits once with the same sequence number, peer ACKs — chat resumes.

Scenario B — connection lost

peer disappears — both retries fail

```
Another test message

[TIMEOUT] Retrying...

[CONNECTION LOST]
```

After the second timeout both programs exit cleanly. To demonstrate live: pull the Wi-Fi on the peer mid-session.

What the .pcapng capture must show

filter: udp.port == <your-derived-port>

No.	Time	Source	Dest	Protocol	Info
1	0.000	client	server	UDP	HELLO-GIU-<LeaderID>-<Sum>
2	0.014	server	client	UDP	OK-GIU-<LeaderID>-<Sum>
3	2.310	client	server	UDP	Hello from the client side
4	2.318	server	client	UDP	ACK:1
5	5.642	server	client	UDP	How are things on your end?
6	5.655	client	server	UDP	ACK:2
7	7.001	client	server	UDP	All good — another one
8	7.012	server	client	UDP	ACK:3

Must be labelled

- HELLO
- OK
- DATA ×3 min
- ACK ×3 min

Payload appears in the Data field as readable UTF-8 — no hex decoding needed.

giunet_session.log — what graders verify

stdout when the program runs

```
+----------------------------------------+
| GIU-Net Session Logger -- ACTIVE       |
| Host:     <your-machine-hostname>      |
| Log:      giunet_session.log           |
| Started:  <real wall-clock timestamp>  |
| Every packet is being recorded.        |
+----------------------------------------+
[GIULogger] #0001 | SEND | seq=0 | "HELLO-..."
[GIULogger] #0002 | RECV | seq=0 | "OK-..."
```

Required of the log file

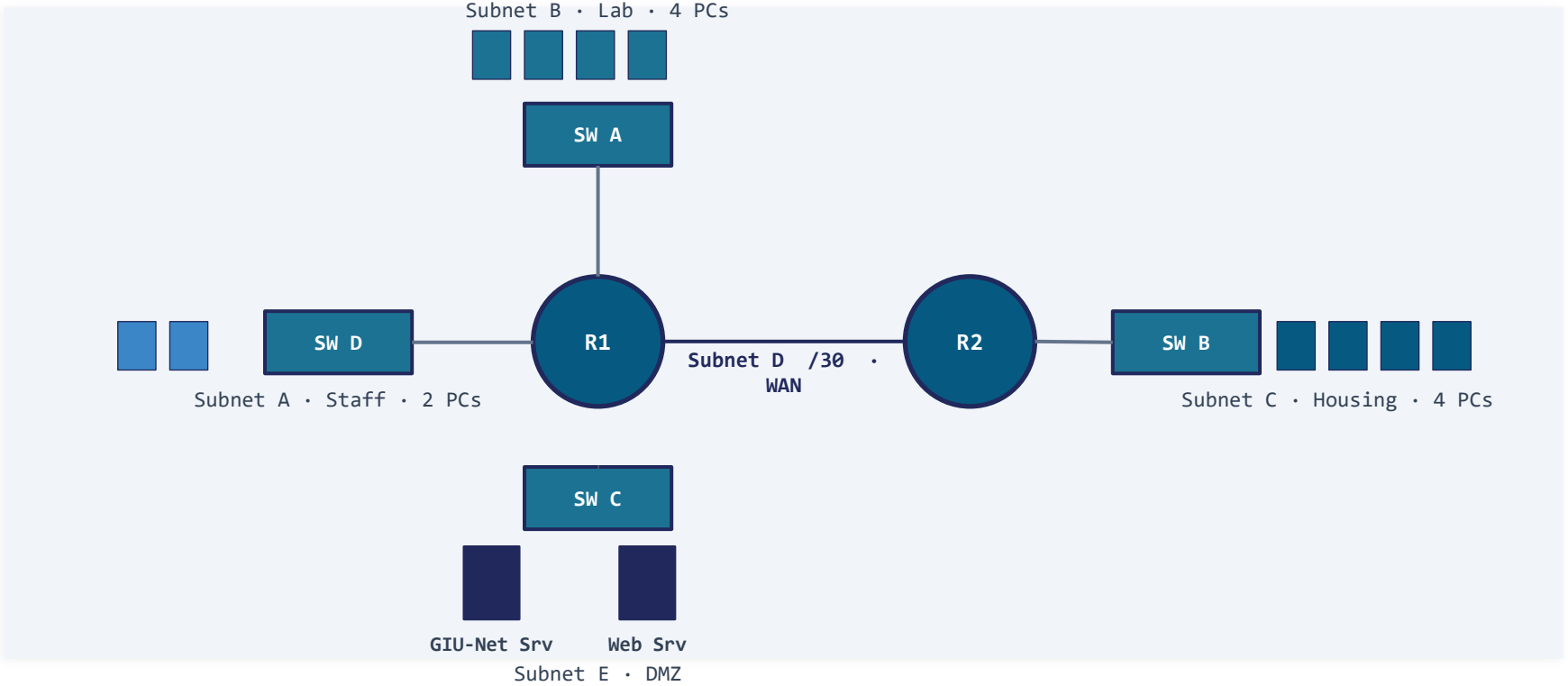
- Binary file in the project root
- Non-zero size
- Generated from a real run on two separate machines (not localhost)
- Contains the HELLO and OK handshake
- Contains at least 5 chat exchanges
- Embedded hostname and timestamps

PART 2

Packet Tracer — Expected Topology & Output

Topology, VLSM table, routing proof, webpage, simulation trace, and the ACL bonus.

What the .pkt file should contain



Subnet table — required columns & order

Subnet	Purpose	Hosts	Prefix	Network	First IP	Last IP	Broadcast
C	Student Housing	200	/24	…	…	…	…
B	Student Lab	100	/25	…	…	…	…
A	Staff Offices	50	/26	…	…	…	…
E	Server DMZ	10	/28	…	…	…	…
D	Router WAN Link	2	/30	…	…	…	…

Allocation order

Largest → smallest subnet, to minimise wasted address space inside the /22 block.

Show your work

Manual binary calculations must be shown in the report. Pasted online-calculator output without working is not accepted.

Cross-subnet ping — verification screenshot

Packet Tracer · Lab/Housing-PC · Command Prompt

```
Housing-PC-1>  ping <GIU-Net Server IP>


Pinging <server-ip> with 32 bytes of data:

Reply from <server-ip>: bytes=32 time<1ms TTL=126

Reply from <server-ip>: bytes=32 time<1ms TTL=126

Reply from <server-ip>: bytes=32 time<1ms TTL=126

Reply from <server-ip>: bytes=32 time<1ms TTL=126


Ping statistics for <server-ip>:

    Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Required proof

- Housing-PC (Subnet C) reaches GIU-Net Server (Subnet E)
- TTL decremented at each router shows the two-hop path
- Captured in Simulation Mode so the hop-by-hop path is visible
- Routing tables for R1 and R2 included in the report

Custom team webpage in Packet Tracer's browser

Requirements

- HTTP service enabled on the Web Server in Subnet E
- index.html edited to show the team name and all 4 members (full name + ID)
- Page loaded from a PC in Subnet B (Lab) using the built-in browser
- Screenshot clearly readable

Hop-by-hop packet trace — 4 screenshots required

HOP 1

Lab-PC-1

Source

L3

L2

L3: src=Lab-PC IP, dst=Server IP
L2: src=Lab-PC MAC, dst=R1 Fa0/0 MAC

HOP 2

Router 1 · in Fa0/0

L2 strip · L3 route

L3

L2

Frame stripped
Routing decision examined

HOP 3

Router 1 · out Fa1/0

L2 reframe

L3

L2

New L2 frame built
src=R1 Fa1/0 MAC, dst=Server MAC

HOP 4

GIU-Net Server

Destination

L3

L2

Both L2 and L3 visible
Packet reaches its target

Every screenshot must be annotated: source IP · destination IP · which layer is being processed.

ACL behaviour — two demonstration screenshots

FROM HOUSING — must fail

blocked by ACL 101 on R2 Fa0/0

```
Housing-PC-1>  ping <Server-IP>


Request timed out.

Request timed out.

Packets: Sent = 4, Lost = 4 (100% loss)
```

Subnet C → Subnet E is denied at the inbound interface on Router 2.

FROM LAB — must succeed

permitted — all other traffic allowed

```
Lab-PC-1>  ping <Server-IP>


Reply from <Server-IP>: bytes=32 ...

Reply from <Server-IP>: bytes=32 ...

Packets: Sent = 4, Received = 4 (0% loss)
```

Report must include the ACL config and the paragraph linking it to the HELLO / REJECTED logic in Part 1.

GIUNet_<TeamLeaderID>_SS26.zip — contents

JAVA

GIUServer.java

Fully commented source — server side

JAVA

GIUClient.java

Fully commented source — client side

LOG

giunet_session.log

Binary log from a live two-machine run

PCAP

***.pcapng**

Wireshark capture filtered by your UDP port

PKT

***.pkt**

Complete Packet Tracer topology

PDF

Report.pdf

Team info, design choices, debugging story, tables, screenshots